BEAST 3 Overview

Modernising the Build, the Type System, and the Package Ecosystem

BEAST Core Developer Team

2026

Outline

1. Why JPMS (+ Maven)?
2. Strongly Typed Spec System
3. Package System
4. Migrating Your Package

The Big Picture

Three pillars of change in BEAST 3:

1. **JPMS modules**: replace the custom classloader with the standard Java module system — gets us to Java 25
2. **Strongly-typed spec system**: compile-time checked inputs and domain types
3. **New package distribution**: JPMS service discovery, with Maven Central as the recommended (not required) distribution channel

Key insight

JPMS is the **architecture**. Maven is convenient **tooling** — you can build a BEAST 3 package with Gradle, Ant, or anything that produces a modular JAR.

Problems We Wanted to Solve

Classloading & isolation

- Custom classloader predates Java modules by 15+ years
- Source of "magic" that confuses new developers and breaks standard tooling
- Kept us stuck on old Java versions
- Using BEAST as a **library** (e.g. LPhyBEAST) was nearly impossible

Build & dependencies

- Ant `build.xml`: manual JAR management, no transitive dependency resolution
- Adding a library means hand-copying JARs and editing classpaths
- No standard way to publish or consume packages

JPMS is the Architecture, Maven is the Tooling

JPMS (the architecture)

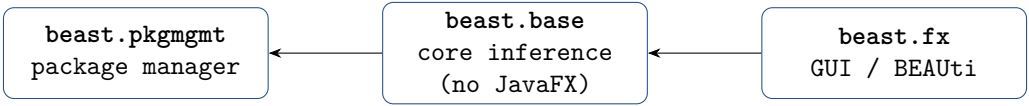
- Replaces the custom classloader with a standard Java mechanism
- Strong encapsulation: modules declare exports
- Per-package `ModuleLayer` isolation
- `module-info.java` `provides` declarations are the single source of truth for service discovery
- Gets us to **Java 25**

Maven (recommended tooling)

- Declared dependencies in `pom.xml`; automatic transitive resolution
- IDE, CI, and Maven Central integration out of the box
- Reproducible builds
- **Not required:** Gradle, Ant, or any tool that produces a modular JAR will work

BEAST 3 Module Architecture

Three JPMS modules, each with its own `module-info.java`:



- `beast-base` has **no JavaFX dependency**: can run headlessly on clusters
- JavaFX is a **Maven dependency**, not bundled with the JDK
- All modules declared as `open module` (reflection OK for XML unmarshalling)

The Type Safety Problem

In BEAST 2, almost everything is `Input<RealParameter>` or `Input<Function>`:

- Type errors only discovered at **runtime**: wrong parameter passed to wrong input
- Bounds (`lower="0"`, `upper="1"`) are stringly-typed XML attributes
- No way to express "this must be a positive real" in the Java API
- IDE cannot help you: everything looks like the same type

Goal

Make invalid models **unrepresentable** at compile time.

Strongly Typed Parameters & Domains

Typed parameters

- `RealScalarParam`: single real
- `RealVectorParam`: vector of reals
- `IntScalarParam` / `IntVectorParam`
- `BoolScalarParam` / `BoolVectorParam`
- `SimplexParam`: probability simplex

Domain types

- `PositiveReal`: replaces `lower="0"`
- `UnitInterval`: replaces `lower="0"` `upper="1"`
- `Real`: unbounded
- `NonNegativeInt`: replaces `lower="0"`
- `PositiveInt`: replaces `lower="1"`

- Domain is part of the **Java type**: compiler catches mismatches
- Legacy classes are **deprecated, not removed**: migrate incrementally

Before / After: Input Declaration

BEAST 2

```
public Input<RealParameter> kappaInput =
    new Input<>("kappa",
    "ts/tv ratio",
    Validate.REQUIRED);

// lower="0" only checked at runtime
// in the XML
```

BEAST 3

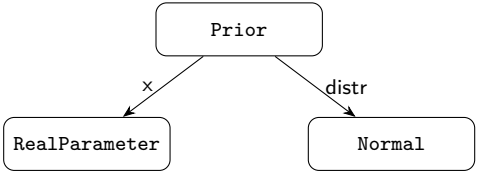
```
public Input<RealScalar<PositiveReal>>
    kappaInput = new Input<>("kappa",
    "ts/tv ratio",
    Validate.REQUIRED);

// PositiveReal is in the type!
// Compiler rejects wrong domain
```

“Distribution IS the Prior”

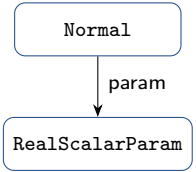
BEAST 2

`Prior` wraps distribution + parameter:



BEAST 3

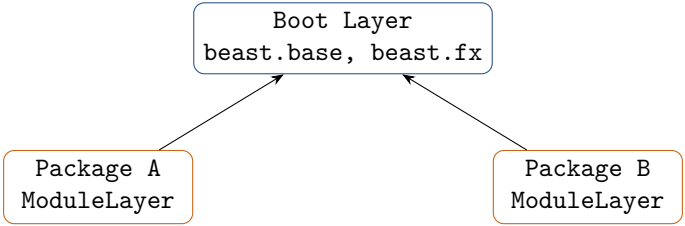
Distribution **is** the prior:



Simpler model graph: no separate `Prior` wrapper needed.

Package System Overview

- **One ModuleLayer per package:** namespace isolation without losing visibility to the core API
- **Same code path for dev and deployment:** `module-info.java` `provides` declarations are the single source of truth
- **Graceful degradation:** modules with unsatisfied dependencies are silently skipped



Package Distribution: Maven Central & ZIP

Maven packages (recommended)

- Published to Maven Central
- Install via BEAUti or CLI
- Automatic dependency resolution
- `maven-packages.xml` tracks installs

ZIP packages (legacy, still works)

- Traditional CBAN archives
- One subdirectory per package
- `version.xml` for service discovery
- Self-describing directory structure

Both formats coexist and both use the **same JPMS `ModuleLayer` loader** at runtime.
Loading precedence: boot layer → Maven → ZIP.

beast-package-skeleton

Ready-to-copy template project on GitHub:

- Maven `pom.xml` with BEAST 3 dependencies
- `module-info.java` with `provides` declarations
- Example distribution, operator, and tests
- BEAST XML using the strongly-typed spec API
- Maven Central release profile

Start here: `https://github.com/CompEvol/beast-package-skeleton`

Step 1: Set Up Your Build (Maven Recommended)

Replace `build.xml` with a `pom.xml` (or use Gradle — any tool that produces a modular JAR):

```xml
<project>
  <groupId>io.github.yourorg</groupId>
  <artifactId>beast-mypackage</artifactId>
  <version>1.0.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>io.github.compevol</groupId>
      <artifactId>beast-base</artifactId>
      <version>2.8.0-SNAPSHOT</version>
    </dependency>
  </dependencies>
</project>
```

- Requires JDK 25+ and Maven 3.9+
- Build BEAST 3 locally first: `mvn install -DskipTests`
- See `beast-package-skeleton` for a complete template POM

Step 2: Add `module-info.java`

Create `src/main/java/module-info.java`:

```
open module my.beast.package {
    requires beast.pkgmgmt;
    requires beast.base;
    requires static beast.fx; // optional at runtime
    requires static javafx.controls; // optional at runtime

    exports my.beast.package;

    provides beast.base.core.BEASTInterface with
        my.beast.package.MyModel,
        my.beast.package.MyOperator,
        my.beast.package.app.beauti.MyInputEditor;
}
```

- Every `BEASTInterface` subclass **must** be listed in `provides`
- `requires static`: dependency is optional at runtime (headless OK)
- GUI classes go in `*.app.beauti` subpackage by convention

Step 3: Fix Compile Errors

Common changes needed when migrating package code:

BEAST 2 pattern	**BEAST 3 replacement**
`StateNode` implements `Function`	No longer a `Function`; use spec types (`RealScalar`, etc.)
`stateNode.scale(s)`	Implement `Scalable` interface
`Prior` + distribution wrapper	Distribution is the prior directly
`Input<RealParameter>`	`Input<RealScalar<PositiveReal>>`
`Evaluator`	Removed; refactor MCMC usage

Legacy classes (`RealParameter`, `Prior`, `Function`) are **deprecated, not removed**. Your package will still compile — migrate to spec types incrementally.

Step 4: Test & Publish

Testing

- `mvn test` runs JUnit tests
- IntelliJ auto-discovers JPMS modules — run/debug XML from IDE
- Namespace test resources under your module to avoid split-package errors

Publishing to Maven Central

- Add release profile to POM (see skeleton)
- Tag release → GitHub Actions auto-publishes JAR
- Users install via BEAUti with automatic dependency resolution

Quick-start checklist

1. Clone `beast-package-skeleton`
2. Rename module, packages, POM coords
3. Copy your source into `src/main/java/`
4. `mvn compile` — fix errors
5. Add classes to `module-info.java` `provides`
6. `mvn test`
7. Update BEAST XML specs if needed

Key Breaking Changes

Area	**BEAST 2**	**BEAST 3**
Modules	Custom classloader	JPMS `ModuleLayer`
Build	Ant, Java 8/11	Maven (recommended), Java 25
Parameters	`RealParameter`	Typed spec params + domains
Priors	`Prior` wrapper	Distribution is the prior
`StateNode`	implements `Function`	No longer a `Function`
`StateNode.scale()`	built-in	Removed; use `Scalable`
JavaFX	Bundled with JDK	Maven dependency

Legacy classes are **deprecated, not removed**: migrate incrementally.

Timeline

Now	Package migration	**Release + paper**	ISBA
Mar 2026	Apr–May	26 Jun 2026	26 Jun–5 Jul
Stakeholders meeting (19 Mar)	20+ packages across multiple groups	Before SMBE Copenhagen	

Resources

Package skeleton `https://github.com/CompEvol/beast-package-skeleton`

Migration guide `scripts/migration-guide.md`: class mapping tables, detailed instructions

CHANGELOG `CHANGELOG.md`: full list of what changed and why

BEAST 3 repo `https://github.com/CompEvol/beast3`

Getting help

Clone the skeleton, copy your source in, and start fixing compile errors. Most packages should compile with only minor changes to deprecated APIs.